

rolling_ball_td3

May 20, 2026

1 Projekt 5 - Przestrzenie Ciągłe

1.1 Autorzy:

- Patrick Bajorski
- Jan Banasik

1.2 1. Opis środowiska i algorytmu

Środowisko - RollingBallEnv. Kulka (promień 12 px) toczy się po planszy 600×600 px z sześcioma prostokątnymi przeszkodami. Agent obserwuje 13 wartości ciągłych: znormalizowaną prędkość (`vel_x`, `vel_y`), jednostkowy wektor kierunku do celu (`goal_dx`, `goal_dy`), znormalizowaną odległość do celu (`goal_dist`) oraz odczyty 8 promieni raycasting rozmieszczonych co 45°, mierzących odległość do najbliższej ściany lub przeszkody. Przestrzeń akcji jest ciągła - dwuwymiarowy wektor siły `Box([-1,-1], [1,1])` przykładanej do kulki. Epizod kończy się sukcesem (`terminated`), gdy kulka dotknie celu, lub po 500 krokach (`truncated`). Kształtowanie nagrody: $-0.01 + 0.02 \cdot (\text{prev_dist} - \text{dist})$ za każdy krok plus $+50$ za dotarcie do celu.

Algorytm - TD3. TD3 (Twin Delayed Deep Deterministic Policy Gradient, Fujimoto i in. 2018) jest algorytmem off-policy przeznaczonym wyłącznie do ciągłych przestrzeni akcji. Wprowadza trzy usprawnienia względem DDPG: - **Twin Critics** - dwa niezależne krytycy Q1 i Q2; do obliczenia celu Bellmana używane jest minimum z ich oszacowań, co eliminuje optymistyczne przeszacowanie wartości Q - **Delayed Policy Update** - aktor aktualizowany co `policy_delay` kroków krytyków, co stabilizuje parę aktor-krytycy - **Target Policy Smoothing** - do docelowej akcji dodawany jest clippowany szum Gaussa, co wygładza powierzchnię wartości i zmniejsza podatność na strome lokalne maksima. Agent zbiera przejścia (`s, a, r, s`) do buforu replay i losowo próbkuję mini-batche do aktualizacji sieci.

```
[6]: import os
import sys
import time
import warnings
warnings.filterwarnings('ignore')

if '.' not in sys.path:
    sys.path.insert(0, '.')

import numpy as np
import matplotlib.pyplot as plt
```

```

from rolling_ball_env import RollingBallEnv
from stable_baselines3 import TD3
from stable_baselines3.common.monitor import Monitor
from stable_baselines3.common.callbacks import EvalCallback

os.makedirs('logs', exist_ok=True)

TOTAL_TIMESTEPS = 200_000
EVAL_FREQ       = 10_000
N_EVAL_EPS      = 10
N_SEEDS         = 10

print('Importy OK.')
print(f'TOTAL_TIMESTEPS={TOTAL_TIMESTEPS:,}, EVAL_FREQ={EVAL_FREQ:,},  

      ↪N_SEEDS={N_SEEDS:,}')

```

Importy OK.

TOTAL_TIMESTEPS=200,000, EVAL_FREQ=10,000, N_SEEDS=10

```

[ ]: # Pomiar czasu kroku i epizodu
_env = RollingBallEnv()
_obs, _ = _env.reset(seed=42)

_N = 5_000
_t0 = time.perf_counter()
for _ in range(_N):
    _a = _env.action_space.sample()
    _obs, _, _d, _tr, _ = _env.step(_a)
    if _d or _tr:
        _obs, _ = _env.reset()
_step_us = (time.perf_counter() - _t0) / _N * 1e6

_N_EP = 200
_ep_lens = []
_t0 = time.perf_counter()
for _ in range(_N_EP):
    _obs, _ = _env.reset()
    _n = 0
    while True:
        _obs, _, _d, _tr, _ = _env.step(_env.action_space.sample())
        _n += 1
        if _d or _tr:
            break
    _ep_lens.append(_n)
_ep_ms = (time.perf_counter() - _t0) / _N_EP * 1e3
_env.close()

```

```

print(f'Czas jednego kroku:    {_step_us:.2f} μs  ({1e6/_step_us:.0f} kroków/
↪s)')
print(f'Czas jednego epizodu:  {_ep_ms:.2f} ms  (śr. {np.mean(_ep_lens):.0f}
↪kroków/epizod, losowe akcje)')
print(f'Szac. czas 1 seeda:    ~{TOTAL_TIMESTEPS * _step_us / 1e6:.0f} s  (sam
↪krok, bez narzutu SB3/ewaluacji)')

```

Czas jednego kroku: 120.64 μs (8289 kroków/s)
 Czas jednego epizodu: 60.10 ms (śr. 500 kroków/epizod, losowe akcje)
 Szac. czas 1 seeda: ~24 s (sam krok, bez narzutu SB3/ewaluacji)

1.3 2. Porównanie zestawów hiperparametrów

Testujemy trzy konfiguracje różniące się **współczynnikiem uczenia i rozmiarem mini-batcha**. Pozostałe parametry są stałe: `gamma=0.99`, `tau=0.005`, `policy_delay=2`, `learning_starts=10 000`, `buffer_size=300 000`, `net_arch=[256, 256]`. Każda konfiguracja uruchamiana **10 razy** z różnymi seedami (0–9), każde uruchomienie przez **200 000 kroków**. Ewaluacja co 10 000 kroków.

Konfiguracja	learning_rate	batch_size	Charakterystyka
A - Baseline	3×10^{-4}	256	Domyślne wartości zbliżone do zaleceń autorów TD3
B - Wysoki LR	10^{-3}	256	3× szybsze aktualizacje wag; ryzyko niestabilności krytyków
C - Niski LR + mały batch	10^{-5}	128	Zachowawcze aktualizacje; wolniejsza konwergencja

```

[ ]: def run_experiment(config_name, model_kwargs, log_base='logs',
                        n_seeds=N_SEEDS, total_timesteps=TOTAL_TIMESTEPS):
    config_dir = os.path.join(log_base, config_name)
    os.makedirs(config_dir, exist_ok=True)
    all_results = []

    expected_evals = total_timesteps // EVAL_FREQ

    for seed in range(n_seeds):
        seed_dir = os.path.join(config_dir, f'seed_{seed}')
        eval_path = os.path.join(seed_dir, 'evaluations.npz')

        if os.path.exists(eval_path):
            cached = dict(np.load(eval_path))

```

```

        if len(cached['timesteps']) >= expected_evals:
            all_results.append(cached)
            print(f' [{config_name}] seed {seed}: wczytano z cache')
            continue
        else:
            print(f' [{config_name}] seed {seed}: cache nieaktualne '
                  f'({len(cached["timesteps"])} != {expected_evals} ewal.),
↳retrenuje')

    os.makedirs(seed_dir, exist_ok=True)
    train_env = RollingBallEnv()
    eval_env = Monitor(RollingBallEnv())

    model = TD3('MlpPolicy', train_env, seed=seed, verbose=0,
↳**model_kwargs)
    cb = EvalCallback(
        eval_env,
        best_model_save_path=seed_dir,
        log_path=seed_dir,
        eval_freq=EVAL_FREQ,
        n_eval_episodes=N_EVAL_EPS,
        deterministic=True,
        render=False,
    )
    model.learn(total_timesteps=total_timesteps, callback=cb)
    train_env.close()
    eval_env.close()

    all_results.append(dict(np.load(eval_path)))
    print(f' [{config_name}] seed {seed}: done')

    return all_results

def aggregate(results_list):
    arrays = [r['results'].mean(axis=1) for r in results_list]
    min_len = min(len(a) for a in arrays)
    if min_len < len(arrays[0]):
        print(f' [aggregate] uwaga: różne długości checkpointów - przycinanie
↳do {min_len}')
    ts = results_list[0]['timesteps'][:min_len]
    per_seed = np.array([a[:min_len] for a in arrays])
    return ts, per_seed.mean(axis=0), per_seed.std(axis=0)

def plot_curves(configs_data, labels, title, save_path=None):
    fig, ax = plt.subplots(figsize=(11, 5))

```

```

colors = plt.cm.tab10.colors
for i, (data, label) in enumerate(zip(configs_data, labels)):
    ts, mean, std = aggregate(data)
    ax.plot(ts, mean, label=label, color=colors[i], linewidth=1.8)
    ax.fill_between(ts, mean - std, mean + std, alpha=0.18, color=colors[i])
ax.axhline(0, color='gray', linewidth=0.7, linestyle='--')
ax.set_xlabel('Kroki (timesteps)')
ax.set_ylabel('Nagroda epizodu (śr. z 10 ep. ewaluacyjnych)')
ax.set_title(title)
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
if save_path:
    plt.savefig(save_path, dpi=150)
plt.show()

```

```

[9]: COMMON = dict(
    gamma=0.99, tau=0.005, policy_delay=2,
    learning_starts=10_000, buffer_size=300_000,
    policy_kwargs={'net_arch': [256, 256]},
)

configs = {
    'A_baseline':      {**COMMON, 'learning_rate': 3e-4, 'batch_size': 256},
    'B_wysoki_lr':     {**COMMON, 'learning_rate': 1e-3, 'batch_size': 256},
    'C_niski_lr_batch': {**COMMON, 'learning_rate': 1e-4, 'batch_size': 128},
}

print(f'Porównanie hiperparametrów: {len(configs)} konfiguracje × {N_SEEDS}┐
    ↪seedów × {TOTAL_TIMESTEPS:,} kroków')
print(f'Łącznie {len(configs) * N_SEEDS * TOTAL_TIMESTEPS / 1e6:.1f} M┐
    ↪kroków\n')

results = {}
for name, kwargs in configs.items():
    print(f'--- Konfiguracja {name} ---')
    results[name] = run_experiment(name, kwargs)
    print()

print('Trening zakończony.')

```

Porównanie hiperparametrów: 3 konfiguracje × 10 seedów × 200,000 kroków
 Łącznie 6.0 M kroków

--- Konfiguracja A_baseline ---

[A_baseline] seed 0: cache nieaktualne (2 != 20 ewal.), retrenuje
 Eval num_timesteps=10000, episode_reward=-5.19 +/- 0.84

Episode length: 500.00 +/- 0.00
New best mean reward!
Eval num_timesteps=20000, episode_reward=20.99 +/- 28.47
Episode length: 367.70 +/- 164.39
New best mean reward!
Eval num_timesteps=30000, episode_reward=2.04 +/- 17.36
Episode length: 462.20 +/- 113.40
Eval num_timesteps=40000, episode_reward=32.15 +/- 28.73
Episode length: 280.70 +/- 181.24
New best mean reward!
Eval num_timesteps=50000, episode_reward=14.06 +/- 25.92
Episode length: 372.90 +/- 194.55
Eval num_timesteps=60000, episode_reward=48.41 +/- 16.24
Episode length: 181.60 +/- 142.11
New best mean reward!
Eval num_timesteps=70000, episode_reward=32.66 +/- 27.44
Episode length: 261.60 +/- 196.07
Eval num_timesteps=80000, episode_reward=42.66 +/- 24.36
Episode length: 198.20 +/- 161.93
Eval num_timesteps=90000, episode_reward=55.02 +/- 2.00
Episode length: 125.80 +/- 52.74
New best mean reward!
Eval num_timesteps=100000, episode_reward=37.86 +/- 24.30
Episode length: 215.60 +/- 191.54
Eval num_timesteps=110000, episode_reward=49.45 +/- 16.74
Episode length: 175.00 +/- 123.01
Eval num_timesteps=120000, episode_reward=55.27 +/- 2.69
Episode length: 132.50 +/- 85.74
New best mean reward!
Eval num_timesteps=130000, episode_reward=48.42 +/- 17.34
Episode length: 144.00 +/- 129.43
Eval num_timesteps=140000, episode_reward=43.64 +/- 22.54
Episode length: 197.10 +/- 165.21
Eval num_timesteps=150000, episode_reward=54.76 +/- 2.81
Episode length: 105.10 +/- 49.24
Eval num_timesteps=160000, episode_reward=43.54 +/- 24.14
Episode length: 184.80 +/- 161.69
Eval num_timesteps=170000, episode_reward=44.53 +/- 22.43
Episode length: 187.40 +/- 163.41
Eval num_timesteps=180000, episode_reward=50.46 +/- 18.41
Episode length: 161.80 +/- 120.47
Eval num_timesteps=190000, episode_reward=49.09 +/- 17.46
Episode length: 128.20 +/- 127.86
Eval num_timesteps=200000, episode_reward=54.01 +/- 1.54
Episode length: 105.00 +/- 68.20
[A_baseline] seed 0: done
[A_baseline] seed 1: wczytano z cache
[A_baseline] seed 2: wczytano z cache

```
[A_baseline] seed 3: wczytano z cache
[A_baseline] seed 4: wczytano z cache
[A_baseline] seed 5: wczytano z cache
[A_baseline] seed 6: wczytano z cache
[A_baseline] seed 7: wczytano z cache
[A_baseline] seed 8: wczytano z cache
[A_baseline] seed 9: wczytano z cache
```

--- Konfiguracja B_wysoki_lr ---

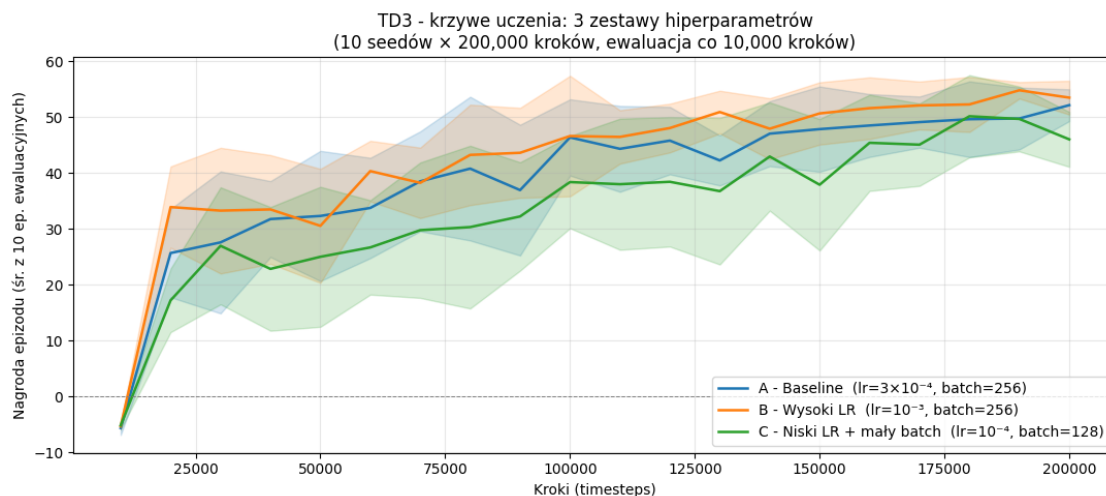
```
[B_wysoki_lr] seed 0: wczytano z cache
[B_wysoki_lr] seed 1: wczytano z cache
[B_wysoki_lr] seed 2: wczytano z cache
[B_wysoki_lr] seed 3: wczytano z cache
[B_wysoki_lr] seed 4: wczytano z cache
[B_wysoki_lr] seed 5: wczytano z cache
[B_wysoki_lr] seed 6: wczytano z cache
[B_wysoki_lr] seed 7: wczytano z cache
[B_wysoki_lr] seed 8: wczytano z cache
[B_wysoki_lr] seed 9: wczytano z cache
```

--- Konfiguracja C_niski_lr_batch ---

```
[C_niski_lr_batch] seed 0: wczytano z cache
[C_niski_lr_batch] seed 1: wczytano z cache
[C_niski_lr_batch] seed 2: wczytano z cache
[C_niski_lr_batch] seed 3: wczytano z cache
[C_niski_lr_batch] seed 4: wczytano z cache
[C_niski_lr_batch] seed 5: wczytano z cache
[C_niski_lr_batch] seed 6: wczytano z cache
[C_niski_lr_batch] seed 7: wczytano z cache
[C_niski_lr_batch] seed 8: wczytano z cache
[C_niski_lr_batch] seed 9: wczytano z cache
```

Trening zakończony.

```
[10]: plot_curves(
    [results['A_baseline'], results['B_wysoki_lr'],
    ↪ results['C_niski_lr_batch']],
    [
        'A - Baseline (lr=3×10-3, batch=256)',
        'B - Wysoki LR (lr=10-3, batch=256)',
        'C - Niski LR + mały batch (lr=10-3, batch=128)',
    ],
    f'TD3 - krzywe uczenia: 3 zestawy hiperparametrów\n'
    f'({N_SEEDS} seedów × {TOTAL_TIMESTEPS:,} kroków, ewaluacja co {EVAL_FREQ:
    ↪:,} kroków)',
    save_path='hyperparams_curves.png',
)
```



1.3.1 Analiza wyników - hiperparametry

Konfiguracja **A (Baseline, $lr=3 \times 10^{-4}$, batch=256)** wykazuje stabilną, monotoniczną konwergencję: średnia nagroda rośnie od 32.3 ± 11.7 przy 50 000 kroków do 46.4 ± 6.8 przy 100 000 kroków i 52.1 ± 2.8 przy 200 000 kroków. Najniższe końcowe odchylenie standardowe ($=2.8$) spośród wszystkich trzech konfiguracji świadczy o wysokiej powtarzalności - wszystkie 10 seedów zbiegło do podobnego poziomu wydajności.

Konfiguracja **B (Wysoki LR, $lr=10^{-3}$, batch=256)** osiąga przy 50 000 kroków zbliżoną nagrodę do A (30.6 ± 10.2 vs 32.3 ± 11.7), a więc wbrew intuicji nie konwerguje wyraźnie szybciej we wczesnej fazie. Przy 100 000 kroków widoczna jest jednak wyższa niestabilność: odchylenie standardowe wynosi 10.8 (vs 6.8 dla A), co wskazuje na silniejsze różnice między seedami. Końcowa nagroda 53.5 ± 3.0 jest porównywalna z A, lecz w obrębie błędu statystycznego - obie konfiguracje mieszczą się w tej samej klasie wydajności.

Konfiguracja **C (Niski LR + mały batch, $lr=10^{-4}$, batch=128)** jest wyraźnie najslabsza w całym zakresie treningu: 25.0 ± 12.5 przy 50 000 kroków, 38.4 ± 8.3 przy 100 000 kroków i 46.0 ± 4.9 przy 200 000 kroków. Nawet po 200 000 krokach nie osiąga poziomu, który A uzyskuje już przy 100 000 kroków. Mniejszy batch wprowadza więcej szumu do gradientów, a zbyt mały krok uczenia spowalnia konwergencję.

Wniosek: konfiguracja **A** jest najlepsza z perspektywy stabilności (najniższe $=2.8$ na końcu treningu) i spójności konwergencji. Konfiguracja **B** osiąga podobną końcową nagrodę, lecz przy wyższej wariancji między seedami. Konfiguracja **C** jest wyraźnie gorsza - przy tym samym budżecie kroków osiąga o ~ 6 punktów niższą nagrodę niż A.

1.4 3. Porównanie architektur sieci

TD3 utrzymuje cztery sieci MLP: **aktor** (deterministyczna polityka), **krytycy Q1 i Q2** (funkcje wartości akcji) - każdy z parą sieci aktualną i docelową (target network). Do porównania architektur używamy hiperparametrów konfiguracji **A ($lr=3 \times 10^{-4}$, batch=256)**, która w sekcji 2 osiągnęła najlepszą stabilność konwergencji ($=2.8$ przy 200 000 kroków). Konfiguracja B osiągnęła zbliżoną

nagrodę końcową, lecz wyższa wariancja między seedami czyni A lepszym punktem odniesienia dla izolowania wpływu architektury.

1.4.1 Architektura Mała - [64, 64] (~16 tys. parametrów łącznie)

Aktor - wejście → wyjście: polityka deterministyczna

Warstwa	Rozmiar	Aktywacja
Wejście	13 (<i>vel_x, vel_y, goal_dx, goal_dy, goal_dist, ray ...ray</i>)	-
Dense	64	ReLU
Dense	64	ReLU
Wyjście	2 (<i>fx, fy</i>)	tanh → akcja $[-1, 1]^2$

Krytycy Q1, Q2 (identyczna struktura, oddzielne wagi) - wejście → wyjście: wartość Q(s, a)

Warstwa	Rozmiar	Aktywacja
Wejście	15 = obserwacja (13) + akcja (2)	-
Dense	64	ReLU
Dense	64	ReLU
Wyjście	1	liniowa → Q(s, a)

1.4.2 Architektura Duża - [256, 256] (~210 tys. parametrów łącznie)

Aktor:

Warstwa	Rozmiar	Aktywacja
Wejście	13	-
Dense	256	ReLU
Dense	256	ReLU
Wyjście	2	tanh → akcja $[-1, 1]^2$

Krytycy Q1, Q2:

Warstwa	Rozmiar	Aktywacja
Wejście	15	-
Dense	256	ReLU
Dense	256	ReLU
Wyjście	1	liniowa → Q(s, a)

Architektura duża ma $\sim 13\times$ więcej parametrów. Większa pojemność sieciowa może precyzyjniej aproksymować złożone zależności między wektorem raycasting a optymalną polityką, lecz wymaga więcej próbek do konwergencji.

```
[12]: # Architektura duża [256,256] - reuse danych z konfiguracji A_baseline
arch_results = {'arch_large_256x256': results['A_baseline']}
print('arch_large_256x256: dane z konfiguracji A_baseline (reuse, bez ponownego
      ↪treningu)')

# Architektura mała [64,64] - nowy eksperyment
arch_small_kwargs = {
    **COMMON,
    'learning_rate': 3e-4,
    'batch_size':    256,
    'policy_kwargs': {'net_arch': [64, 64]},
}
print('\n--- arch_small_64x64 ---')
arch_results['arch_small_64x64'] = run_experiment('arch_small_64x64',
      ↪arch_small_kwargs)

print('\nGotowe.')
```

arch_large_256x256: dane z konfiguracji A_baseline (reuse, bez ponownego treningu)

```
--- arch_small_64x64 ---
[arch_small_64x64] seed 0: cache nieaktualne (1 != 20 ewal.), retrenuje
Eval num_timesteps=10000, episode_reward=-4.95 +/- 1.61
Episode length: 500.00 +/- 0.00
New best mean reward!
Eval num_timesteps=20000, episode_reward=44.28 +/- 23.36
Episode length: 231.10 +/- 144.83
New best mean reward!
Eval num_timesteps=30000, episode_reward=25.50 +/- 28.31
Episode length: 285.10 +/- 216.67
Eval num_timesteps=40000, episode_reward=26.06 +/- 27.98
Episode length: 285.80 +/- 215.42
Eval num_timesteps=50000, episode_reward=32.73 +/- 28.55
Episode length: 268.80 +/- 190.92
Eval num_timesteps=60000, episode_reward=25.57 +/- 27.30
Episode length: 276.60 +/- 223.73
Eval num_timesteps=70000, episode_reward=41.98 +/- 22.91
Episode length: 153.80 +/- 175.99
Eval num_timesteps=80000, episode_reward=50.38 +/- 17.73
Episode length: 162.90 +/- 121.50
New best mean reward!
Eval num_timesteps=90000, episode_reward=49.12 +/- 17.40
Episode length: 166.70 +/- 136.51
```

Eval num_timesteps=100000, episode_reward=55.46 +/- 2.14
 Episode length: 138.10 +/- 60.40
 New best mean reward!
 Eval num_timesteps=110000, episode_reward=44.44 +/- 22.30
 Episode length: 186.80 +/- 164.27
 Eval num_timesteps=120000, episode_reward=54.11 +/- 2.04
 Episode length: 66.40 +/- 25.78
 Eval num_timesteps=130000, episode_reward=48.00 +/- 17.32
 Episode length: 124.00 +/- 135.88
 Eval num_timesteps=140000, episode_reward=43.70 +/- 22.76
 Episode length: 185.60 +/- 159.58
 Eval num_timesteps=150000, episode_reward=48.61 +/- 16.13
 Episode length: 134.40 +/- 131.62
 Eval num_timesteps=160000, episode_reward=49.27 +/- 18.61
 Episode length: 146.20 +/- 132.99
 Eval num_timesteps=170000, episode_reward=43.90 +/- 23.49
 Episode length: 188.60 +/- 158.50
 Eval num_timesteps=180000, episode_reward=55.24 +/- 2.30
 Episode length: 90.90 +/- 44.77
 Eval num_timesteps=190000, episode_reward=54.45 +/- 1.65
 Episode length: 96.80 +/- 47.23
 Eval num_timesteps=200000, episode_reward=49.49 +/- 19.14
 Episode length: 161.10 +/- 121.00
 [arch_small_64x64] seed 0: done
 Eval num_timesteps=10000, episode_reward=-5.86 +/- 1.11
 Episode length: 500.00 +/- 0.00
 New best mean reward!
 Eval num_timesteps=20000, episode_reward=38.67 +/- 25.97
 Episode length: 273.40 +/- 159.69
 New best mean reward!
 Eval num_timesteps=30000, episode_reward=9.87 +/- 22.26
 Episode length: 420.30 +/- 159.44
 Eval num_timesteps=40000, episode_reward=14.31 +/- 27.09
 Episode length: 392.90 +/- 168.80
 Eval num_timesteps=50000, episode_reward=27.69 +/- 28.41
 Episode length: 305.30 +/- 197.60
 Eval num_timesteps=60000, episode_reward=32.81 +/- 28.56
 Episode length: 274.80 +/- 188.33
 Eval num_timesteps=70000, episode_reward=31.50 +/- 28.54
 Episode length: 262.00 +/- 198.58
 Eval num_timesteps=80000, episode_reward=48.67 +/- 16.70
 Episode length: 134.30 +/- 136.78
 New best mean reward!
 Eval num_timesteps=90000, episode_reward=44.82 +/- 23.68
 Episode length: 209.80 +/- 146.76
 Eval num_timesteps=100000, episode_reward=48.99 +/- 17.57
 Episode length: 145.70 +/- 133.95
 New best mean reward!

Eval num_timesteps=110000, episode_reward=50.34 +/- 18.37
 Episode length: 157.60 +/- 123.47
 New best mean reward!
 Eval num_timesteps=120000, episode_reward=54.68 +/- 2.25
 Episode length: 78.40 +/- 38.33
 New best mean reward!
 Eval num_timesteps=130000, episode_reward=55.61 +/- 2.62
 Episode length: 101.00 +/- 51.30
 New best mean reward!
 Eval num_timesteps=140000, episode_reward=51.20 +/- 16.40
 Episode length: 156.90 +/- 118.17
 Eval num_timesteps=150000, episode_reward=49.68 +/- 18.00
 Episode length: 145.20 +/- 129.46
 Eval num_timesteps=160000, episode_reward=43.47 +/- 22.91
 Episode length: 169.90 +/- 169.14
 Eval num_timesteps=170000, episode_reward=54.85 +/- 2.09
 Episode length: 90.00 +/- 32.04
 Eval num_timesteps=180000, episode_reward=48.76 +/- 17.22
 Episode length: 130.50 +/- 130.37
 Eval num_timesteps=190000, episode_reward=54.95 +/- 1.90
 Episode length: 99.30 +/- 40.38
 Eval num_timesteps=200000, episode_reward=55.75 +/- 2.64
 Episode length: 109.40 +/- 44.22
 New best mean reward!
 [arch_small_64x64] seed 1: done
 Eval num_timesteps=10000, episode_reward=-6.33 +/- 0.79
 Episode length: 500.00 +/- 0.00
 New best mean reward!
 Eval num_timesteps=20000, episode_reward=8.06 +/- 23.06
 Episode length: 418.70 +/- 163.37
 New best mean reward!
 Eval num_timesteps=30000, episode_reward=30.78 +/- 29.18
 Episode length: 263.80 +/- 200.74
 New best mean reward!
 Eval num_timesteps=40000, episode_reward=18.05 +/- 29.70
 Episode length: 338.70 +/- 198.76
 Eval num_timesteps=50000, episode_reward=19.17 +/- 29.09
 Episode length: 349.30 +/- 186.73
 Eval num_timesteps=60000, episode_reward=41.56 +/- 24.22
 Episode length: 155.30 +/- 174.44
 New best mean reward!
 Eval num_timesteps=70000, episode_reward=36.12 +/- 28.30
 Episode length: 210.70 +/- 193.23
 Eval num_timesteps=80000, episode_reward=25.07 +/- 29.51
 Episode length: 299.10 +/- 205.03
 Eval num_timesteps=90000, episode_reward=42.64 +/- 23.25
 Episode length: 179.20 +/- 165.90
 New best mean reward!

Eval num_timesteps=100000, episode_reward=48.95 +/- 15.65
 Episode length: 140.70 +/- 138.13
 New best mean reward!
 Eval num_timesteps=110000, episode_reward=38.41 +/- 25.44
 Episode length: 225.40 +/- 184.12
 Eval num_timesteps=120000, episode_reward=42.94 +/- 23.74
 Episode length: 166.50 +/- 169.04
 Eval num_timesteps=130000, episode_reward=50.43 +/- 17.38
 Episode length: 152.50 +/- 125.01
 New best mean reward!
 Eval num_timesteps=140000, episode_reward=49.14 +/- 17.24
 Episode length: 163.80 +/- 131.12
 Eval num_timesteps=150000, episode_reward=38.73 +/- 25.93
 Episode length: 215.80 +/- 188.04
 Eval num_timesteps=160000, episode_reward=49.04 +/- 18.28
 Episode length: 138.30 +/- 127.28
 Eval num_timesteps=170000, episode_reward=49.01 +/- 16.89
 Episode length: 149.80 +/- 126.33
 Eval num_timesteps=180000, episode_reward=31.14 +/- 29.34
 Episode length: 254.60 +/- 201.56
 Eval num_timesteps=190000, episode_reward=43.53 +/- 23.34
 Episode length: 190.40 +/- 163.41
 Eval num_timesteps=200000, episode_reward=55.07 +/- 2.05
 Episode length: 115.70 +/- 49.87
 New best mean reward!
 [arch_small_64x64] seed 2: done
 Eval num_timesteps=10000, episode_reward=-5.15 +/- 1.74
 Episode length: 500.00 +/- 0.00
 New best mean reward!
 Eval num_timesteps=20000, episode_reward=20.88 +/- 27.27
 Episode length: 348.60 +/- 188.16
 New best mean reward!
 Eval num_timesteps=30000, episode_reward=30.52 +/- 26.81
 Episode length: 228.80 +/- 222.03
 New best mean reward!
 Eval num_timesteps=40000, episode_reward=49.48 +/- 18.02
 Episode length: 169.90 +/- 123.95
 New best mean reward!
 Eval num_timesteps=50000, episode_reward=19.92 +/- 29.26
 Episode length: 352.30 +/- 184.85
 Eval num_timesteps=60000, episode_reward=31.61 +/- 30.23
 Episode length: 292.90 +/- 171.96
 Eval num_timesteps=70000, episode_reward=42.07 +/- 22.68
 Episode length: 160.70 +/- 174.51
 Eval num_timesteps=80000, episode_reward=44.29 +/- 23.69
 Episode length: 189.30 +/- 160.97
 Eval num_timesteps=90000, episode_reward=42.73 +/- 24.22
 Episode length: 172.10 +/- 165.96

Eval num_timesteps=100000, episode_reward=44.35 +/- 22.83
 Episode length: 206.40 +/- 152.75
 Eval num_timesteps=110000, episode_reward=48.60 +/- 18.19
 Episode length: 131.30 +/- 132.05
 Eval num_timesteps=120000, episode_reward=48.07 +/- 17.95
 Episode length: 131.00 +/- 132.86
 Eval num_timesteps=130000, episode_reward=42.98 +/- 23.75
 Episode length: 193.30 +/- 158.63
 Eval num_timesteps=140000, episode_reward=37.76 +/- 27.00
 Episode length: 252.70 +/- 182.09
 Eval num_timesteps=150000, episode_reward=48.40 +/- 17.87
 Episode length: 123.80 +/- 128.66
 Eval num_timesteps=160000, episode_reward=42.77 +/- 24.09
 Episode length: 170.40 +/- 169.47
 Eval num_timesteps=170000, episode_reward=43.28 +/- 24.92
 Episode length: 191.50 +/- 157.50
 Eval num_timesteps=180000, episode_reward=55.02 +/- 2.41
 Episode length: 98.70 +/- 61.76
 New best mean reward!
 Eval num_timesteps=190000, episode_reward=50.00 +/- 18.01
 Episode length: 149.70 +/- 123.75
 Eval num_timesteps=200000, episode_reward=54.96 +/- 2.20
 Episode length: 112.50 +/- 71.68
 [arch_small_64x64] seed 3: done
 Eval num_timesteps=10000, episode_reward=-6.13 +/- 1.02
 Episode length: 500.00 +/- 0.00
 New best mean reward!
 Eval num_timesteps=20000, episode_reward=32.20 +/- 28.71
 Episode length: 258.40 +/- 197.77
 New best mean reward!
 Eval num_timesteps=30000, episode_reward=13.03 +/- 28.53
 Episode length: 400.50 +/- 161.70
 Eval num_timesteps=40000, episode_reward=-2.59 +/- 18.13
 Episode length: 454.10 +/- 137.70
 Eval num_timesteps=50000, episode_reward=30.40 +/- 29.54
 Episode length: 243.90 +/- 210.08
 Eval num_timesteps=60000, episode_reward=42.35 +/- 22.39
 Episode length: 188.30 +/- 177.50
 New best mean reward!
 Eval num_timesteps=70000, episode_reward=31.00 +/- 29.43
 Episode length: 266.70 +/- 192.87
 Eval num_timesteps=80000, episode_reward=30.69 +/- 28.88
 Episode length: 274.10 +/- 195.18
 Eval num_timesteps=90000, episode_reward=37.46 +/- 27.14
 Episode length: 230.10 +/- 186.23
 Eval num_timesteps=100000, episode_reward=48.21 +/- 17.89
 Episode length: 175.90 +/- 157.89
 New best mean reward!

Eval num_timesteps=110000, episode_reward=23.35 +/- 28.88
 Episode length: 279.10 +/- 221.67
 Eval num_timesteps=120000, episode_reward=36.02 +/- 27.31
 Episode length: 205.50 +/- 195.73
 Eval num_timesteps=130000, episode_reward=18.89 +/- 29.62
 Episode length: 355.10 +/- 185.73
 Eval num_timesteps=140000, episode_reward=43.73 +/- 24.65
 Episode length: 208.30 +/- 158.77
 Eval num_timesteps=150000, episode_reward=49.59 +/- 17.86
 Episode length: 143.80 +/- 135.85
 New best mean reward!
 Eval num_timesteps=160000, episode_reward=48.79 +/- 18.44
 Episode length: 153.50 +/- 126.98
 Eval num_timesteps=170000, episode_reward=50.56 +/- 16.73
 Episode length: 180.70 +/- 127.90
 New best mean reward!
 Eval num_timesteps=180000, episode_reward=42.94 +/- 23.30
 Episode length: 171.30 +/- 170.15
 Eval num_timesteps=190000, episode_reward=43.52 +/- 23.88
 Episode length: 202.70 +/- 169.40
 Eval num_timesteps=200000, episode_reward=49.70 +/- 18.17
 Episode length: 150.90 +/- 123.34
 [arch_small_64x64] seed 4: done
 Eval num_timesteps=10000, episode_reward=-4.83 +/- 2.12
 Episode length: 500.00 +/- 0.00
 New best mean reward!
 Eval num_timesteps=20000, episode_reward=6.14 +/- 24.45
 Episode length: 430.30 +/- 140.41
 New best mean reward!
 Eval num_timesteps=30000, episode_reward=27.53 +/- 26.60
 Episode length: 297.80 +/- 206.29
 New best mean reward!
 Eval num_timesteps=40000, episode_reward=27.67 +/- 28.44
 Episode length: 308.90 +/- 192.21
 New best mean reward!
 Eval num_timesteps=50000, episode_reward=27.10 +/- 28.93
 Episode length: 302.40 +/- 199.07
 Eval num_timesteps=60000, episode_reward=44.41 +/- 23.61
 Episode length: 195.70 +/- 154.66
 New best mean reward!
 Eval num_timesteps=70000, episode_reward=27.65 +/- 28.19
 Episode length: 311.60 +/- 189.40
 Eval num_timesteps=80000, episode_reward=39.80 +/- 24.82
 Episode length: 234.60 +/- 183.01
 Eval num_timesteps=90000, episode_reward=43.99 +/- 23.11
 Episode length: 184.40 +/- 160.50
 Eval num_timesteps=100000, episode_reward=56.39 +/- 2.89
 Episode length: 130.10 +/- 62.00

New best mean reward!

Eval num_timesteps	episode_reward	episode_reward +/-	Episode length	Episode length +/-
110000	54.18	+/- 1.63	111.90	+/- 50.17
120000	43.38	+/- 24.21	185.60	+/- 162.79
130000	50.64	+/- 17.68	191.10	+/- 132.20
140000	44.37	+/- 23.36	193.60	+/- 157.87
150000	54.14	+/- 1.75	97.20	+/- 81.42
160000	39.45	+/- 25.78	230.60	+/- 179.84
170000	39.71	+/- 25.83	226.50	+/- 181.13
180000	53.68	+/- 2.82	60.00	+/- 43.67
190000	50.02	+/- 18.22	146.80	+/- 130.68
200000	43.83	+/- 24.55	180.20	+/- 165.02

[arch_small_64x64] seed 5: done

Eval num_timesteps	episode_reward	episode_reward +/-	Episode length	Episode length +/-
10000	-2.65	+/- 2.30	500.00	+/- 0.00

New best mean reward!

Eval num_timesteps	episode_reward	episode_reward +/-	Episode length	Episode length +/-
20000	37.24	+/- 25.87	211.10	+/- 189.84

New best mean reward!

Eval num_timesteps	episode_reward	episode_reward +/-	Episode length	Episode length +/-
30000	15.87	+/- 26.34	391.10	+/- 171.71
40000	17.94	+/- 28.67	330.30	+/- 208.84
50000	19.17	+/- 28.95	362.50	+/- 171.46
60000	25.12	+/- 28.16	313.10	+/- 204.07
70000	26.95	+/- 29.09	307.70	+/- 194.89
80000	26.31	+/- 29.07	299.50	+/- 201.95
90000	32.89	+/- 26.31	274.70	+/- 186.42
100000	49.20	+/- 16.69	198.80	+/- 144.86

New best mean reward!

Eval num_timesteps	episode_reward	episode_reward +/-	Episode length	Episode length +/-
110000	33.31	+/- 29.21	270.50	+/- 190.90
120000	32.92	+/- 28.07		

Episode length: 284.90 +/- 186.57
 Eval num_timesteps=130000, episode_reward=31.02 +/- 27.62
 Episode length: 243.40 +/- 211.29
 Eval num_timesteps=140000, episode_reward=43.67 +/- 22.90
 Episode length: 202.50 +/- 162.17
 Eval num_timesteps=150000, episode_reward=37.25 +/- 27.62
 Episode length: 263.50 +/- 181.34
 Eval num_timesteps=160000, episode_reward=50.23 +/- 18.21
 Episode length: 177.50 +/- 123.04
 New best mean reward!
 Eval num_timesteps=170000, episode_reward=43.54 +/- 23.83
 Episode length: 185.80 +/- 160.34
 Eval num_timesteps=180000, episode_reward=43.04 +/- 23.68
 Episode length: 209.10 +/- 154.99
 Eval num_timesteps=190000, episode_reward=43.34 +/- 22.37
 Episode length: 168.20 +/- 170.39
 Eval num_timesteps=200000, episode_reward=50.51 +/- 17.60
 Episode length: 180.30 +/- 126.99
 New best mean reward!
 [arch_small_64x64] seed 6: done
 Eval num_timesteps=10000, episode_reward=-6.02 +/- 3.14
 Episode length: 500.00 +/- 0.00
 New best mean reward!
 Eval num_timesteps=20000, episode_reward=20.07 +/- 28.15
 Episode length: 357.20 +/- 183.76
 New best mean reward!
 Eval num_timesteps=30000, episode_reward=21.17 +/- 27.46
 Episode length: 349.70 +/- 185.76
 New best mean reward!
 Eval num_timesteps=40000, episode_reward=37.17 +/- 26.66
 Episode length: 287.80 +/- 159.47
 New best mean reward!
 Eval num_timesteps=50000, episode_reward=18.70 +/- 28.23
 Episode length: 333.90 +/- 205.94
 Eval num_timesteps=60000, episode_reward=37.90 +/- 25.64
 Episode length: 230.30 +/- 186.69
 New best mean reward!
 Eval num_timesteps=70000, episode_reward=38.70 +/- 25.15
 Episode length: 274.70 +/- 184.69
 New best mean reward!
 Eval num_timesteps=80000, episode_reward=31.72 +/- 29.88
 Episode length: 282.50 +/- 181.02
 Eval num_timesteps=90000, episode_reward=48.32 +/- 17.87
 Episode length: 135.60 +/- 124.84
 New best mean reward!
 Eval num_timesteps=100000, episode_reward=42.62 +/- 23.87
 Episode length: 175.50 +/- 167.84
 Eval num_timesteps=110000, episode_reward=33.47 +/- 26.45

Episode length: 259.40 +/- 198.17
 Eval num_timesteps=120000, episode_reward=33.28 +/- 27.30
 Episode length: 259.60 +/- 200.06
 Eval num_timesteps=130000, episode_reward=55.75 +/- 2.31
 Episode length: 99.90 +/- 41.08
 New best mean reward!
 Eval num_timesteps=140000, episode_reward=54.04 +/- 1.93
 Episode length: 81.10 +/- 37.45
 Eval num_timesteps=150000, episode_reward=55.51 +/- 1.58
 Episode length: 117.80 +/- 53.95
 Eval num_timesteps=160000, episode_reward=49.72 +/- 17.43
 Episode length: 154.20 +/- 126.75
 Eval num_timesteps=170000, episode_reward=55.25 +/- 2.65
 Episode length: 112.20 +/- 55.71
 Eval num_timesteps=180000, episode_reward=56.01 +/- 2.64
 Episode length: 101.60 +/- 36.26
 New best mean reward!
 Eval num_timesteps=190000, episode_reward=48.89 +/- 17.80
 Episode length: 126.90 +/- 130.67
 Eval num_timesteps=200000, episode_reward=47.20 +/- 17.42
 Episode length: 106.70 +/- 134.42
 [arch_small_64x64] seed 7: done
 Eval num_timesteps=10000, episode_reward=-6.11 +/- 1.12
 Episode length: 500.00 +/- 0.00
 New best mean reward!
 Eval num_timesteps=20000, episode_reward=36.65 +/- 26.13
 Episode length: 236.80 +/- 174.36
 New best mean reward!
 Eval num_timesteps=30000, episode_reward=32.99 +/- 27.49
 Episode length: 262.50 +/- 197.91
 Eval num_timesteps=40000, episode_reward=26.27 +/- 28.54
 Episode length: 292.00 +/- 208.86
 Eval num_timesteps=50000, episode_reward=24.99 +/- 28.80
 Episode length: 289.20 +/- 213.46
 Eval num_timesteps=60000, episode_reward=49.43 +/- 17.87
 Episode length: 134.50 +/- 124.64
 New best mean reward!
 Eval num_timesteps=70000, episode_reward=33.18 +/- 27.83
 Episode length: 263.60 +/- 195.95
 Eval num_timesteps=80000, episode_reward=32.56 +/- 26.26
 Episode length: 251.10 +/- 206.53
 Eval num_timesteps=90000, episode_reward=49.53 +/- 16.76
 Episode length: 147.60 +/- 127.08
 New best mean reward!
 Eval num_timesteps=100000, episode_reward=54.45 +/- 1.87
 Episode length: 106.10 +/- 35.18
 New best mean reward!
 Eval num_timesteps=110000, episode_reward=43.77 +/- 24.09

Episode length: 193.30 +/- 157.72
 Eval num_timesteps=120000, episode_reward=55.48 +/- 2.97
 Episode length: 123.90 +/- 62.91
 New best mean reward!
 Eval num_timesteps=130000, episode_reward=55.50 +/- 1.16
 Episode length: 103.90 +/- 31.79
 New best mean reward!
 Eval num_timesteps=140000, episode_reward=54.55 +/- 2.29
 Episode length: 87.60 +/- 51.64
 Eval num_timesteps=150000, episode_reward=54.15 +/- 1.54
 Episode length: 86.20 +/- 38.07
 Eval num_timesteps=160000, episode_reward=50.00 +/- 18.47
 Episode length: 157.80 +/- 127.49
 Eval num_timesteps=170000, episode_reward=50.13 +/- 17.34
 Episode length: 142.30 +/- 125.80
 Eval num_timesteps=180000, episode_reward=50.11 +/- 18.26
 Episode length: 167.30 +/- 123.88
 Eval num_timesteps=190000, episode_reward=49.57 +/- 18.24
 Episode length: 152.60 +/- 123.09
 Eval num_timesteps=200000, episode_reward=48.97 +/- 17.69
 Episode length: 129.90 +/- 128.85
 [arch_small_64x64] seed 8: done
 Eval num_timesteps=10000, episode_reward=-6.25 +/- 1.99
 Episode length: 500.00 +/- 0.00
 New best mean reward!
 Eval num_timesteps=20000, episode_reward=49.51 +/- 17.62
 Episode length: 153.80 +/- 125.61
 New best mean reward!
 Eval num_timesteps=30000, episode_reward=43.81 +/- 23.67
 Episode length: 199.30 +/- 158.00
 Eval num_timesteps=40000, episode_reward=24.88 +/- 29.12
 Episode length: 289.10 +/- 211.85
 Eval num_timesteps=50000, episode_reward=20.41 +/- 27.05
 Episode length: 323.30 +/- 217.14
 Eval num_timesteps=60000, episode_reward=38.43 +/- 27.13
 Episode length: 232.20 +/- 181.49
 Eval num_timesteps=70000, episode_reward=7.59 +/- 24.01
 Episode length: 451.80 +/- 112.81
 Eval num_timesteps=80000, episode_reward=20.88 +/- 28.96
 Episode length: 340.00 +/- 197.01
 Eval num_timesteps=90000, episode_reward=42.71 +/- 23.53
 Episode length: 177.50 +/- 168.44
 Eval num_timesteps=100000, episode_reward=55.15 +/- 2.11
 Episode length: 127.40 +/- 75.26
 New best mean reward!
 Eval num_timesteps=110000, episode_reward=55.21 +/- 2.31
 Episode length: 106.60 +/- 36.84
 New best mean reward!

```

Eval num_timesteps=120000, episode_reward=37.77 +/- 26.78
Episode length: 222.80 +/- 186.14
Eval num_timesteps=130000, episode_reward=48.33 +/- 17.99
Episode length: 136.60 +/- 126.17
Eval num_timesteps=140000, episode_reward=55.27 +/- 2.23
Episode length: 107.70 +/- 45.79
New best mean reward!
Eval num_timesteps=150000, episode_reward=50.36 +/- 17.53
Episode length: 145.60 +/- 123.07
Eval num_timesteps=160000, episode_reward=49.31 +/- 17.87
Episode length: 174.70 +/- 122.37
Eval num_timesteps=170000, episode_reward=55.82 +/- 1.45
Episode length: 91.30 +/- 24.93
New best mean reward!
Eval num_timesteps=180000, episode_reward=49.81 +/- 17.62
Episode length: 149.70 +/- 127.58
Eval num_timesteps=190000, episode_reward=49.39 +/- 17.45
Episode length: 168.60 +/- 137.49
Eval num_timesteps=200000, episode_reward=49.30 +/- 16.82
Episode length: 141.10 +/- 128.32
[arch_small_64x64] seed 9: done

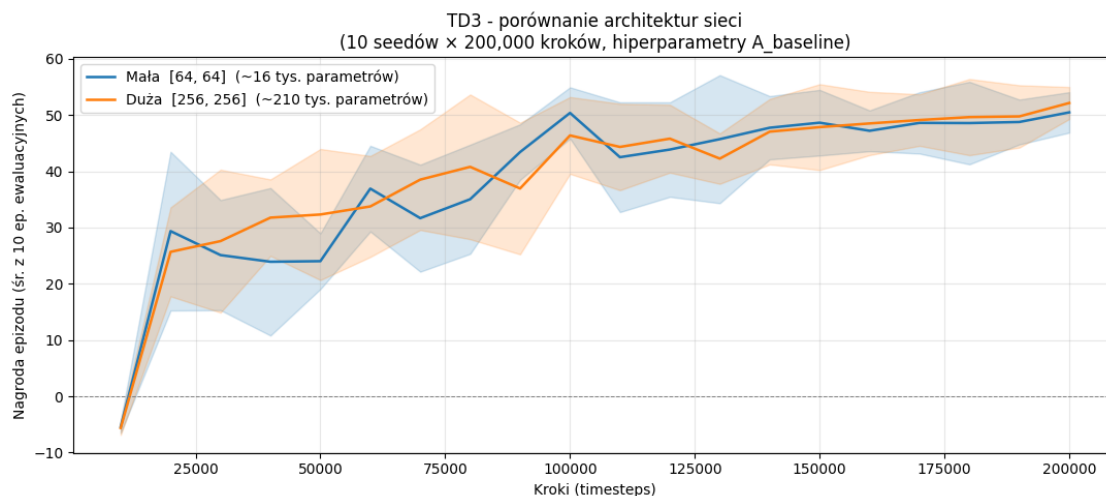
```

Gotowe.

```

[13]: plot_curves(
    [arch_results['arch_small_64x64'], arch_results['arch_large_256x256']],
    [
        'Mała [64, 64] (~16 tys. parametrów)',
        'Duża [256, 256] (~210 tys. parametrów)',
    ],
    f'TD3 - porównanie architektur sieci\n'
    f'({N_SEEDS} seedów × {TOTAL_TIMESTEPS:,} kroków, hiperparametry_
↪A_baseline)',
    save_path='architectures_curves.png',
)

```



1.4.3 Analiza wyników - architektury sieci

Architektura **mała** [64, 64] wykazuje zaskakująco szybką konwergencję w środkowej fazie treningu: przy 100 000 kroków osiąga średnią nagrodę 50.4 ± 4.6 , przewyższając architekturę dużą (46.4 ± 6.8) w tym samym punkcie. Wynik ten można tłumaczyć mniejszą liczbą parametrów - sieć szybciej dopasowuje się do kluczowych wzorców nawigacyjnych, nie marnując pojemności na zbędne zależności. Końcowa nagroda po 200 000 kroków wynosi jednak tylko 50.5 ± 3.6 .

Architektura **duża** [256, 256] startuje wolniej (32.3 ± 11.7 przy 50 000 kroków vs 24.0 ± 5.0 dla małej), lecz z biegiem treningu w pełni wykorzystuje swoją pojemność reprezentacyjną i osiąga końcową nagrodę 52.1 ± 2.8 - o 1.6 punktu wyższą niż mała architektura, przy niższym odchyleniu standardowym (2.8 vs 3.6). Mniejsza wariancja między seedami świadczy o bardziej stabilnej i powtarzalnej konwergencji.

Wniosek: architektura **duża** [256, 256] jest nieznacznie lepsza pod względem końcowej nagrody i stabilności. Różnica jest jednak mała ($\sim 3\%$), co sugeruje, że zadanie nawigacyjne ze 13-wymiarową obserwacją nie wymaga bardzo dużej sieci. Architektura mała ma przewagę szybszej konwergencji w budżecie do ~ 120 000 kroków, co może być istotne przy ograniczonym czasie treningu.

1.5 4. Najlepszy agent - symulacja deterministyczna

Spośród wszystkich przebadanych konfiguracji i seedów (3 zestawy hiperparametrów $\times$ 10 seedów + 1 architektura $\times$ 10 seedów) wybieramy model z najwyższą średnią nagrodą ewaluacyjną zarejestrowaną w trakcie treningu. Okazał się nim model z konfiguracji **B** (Wysoki LR, $\text{lr}=10^{-3}$, **batch=256**), **seed 8**, wykorzystujący architekturę [256, 256] (duże sieci aktora i krytyków), który przy 10-epizodowej ewaluacji treningowej osiągnął średnią nagrodę **56.63**.

Wczytujemy `best_model.zip` zapisany przez `EvalCallback` i uruchamiamy agenta deterministycznie (bez szumu eksploracyjnego), testując na 30 nowych epizodach. Następnie porównujemy uzyskane nagrody z krzywą uczenia konfiguracji B.

```

[ ]: best_reward = -np.inf
best_config = None
best_seed = None
best_source = None

for config_name, seeds_data in results.items():
    for seed_idx, data in enumerate(seeds_data):
        max_r = data['results'].mean(axis=1).max()
        if max_r > best_reward:
            best_reward = max_r
            best_config = config_name
            best_seed = seed_idx
            best_source = 'results'

for config_name, seeds_data in arch_results.items():
    for seed_idx, data in enumerate(seeds_data):
        max_r = data['results'].mean(axis=1).max()
        if max_r > best_reward:
            best_reward = max_r
            best_config = config_name
            best_seed = seed_idx
            best_source = 'arch_results'

arch_map = {
    'A_baseline': '[256, 256]',
    'B_wysoki_lr': '[256, 256]',
    'C_niski_lr_batch': '[256, 256]',
    'arch_large_256x256': '[256, 256]',
    'arch_small_64x64': '[64, 64]',
}

best_model_path = os.path.join('logs', best_config, f'seed_{best_seed}',
    ↪'best_model')
print(f'Najlepszy model:')
print(f' Konfiguracja: {best_config}')
print(f' Architektura sieci: {arch_map[best_config]}')
print(f' Seed: {best_seed}')
print(f' Max nagroda ewal.: {best_reward:.2f}')

```

```

Najlepszy model:
Konfiguracja: B_wysoki_lr
Architektura sieci: [256, 256]
Seed: 8
Max nagroda ewal.: 56.63

```

```

[ ]: N_DET = 30
model = TD3.load(best_model_path)

```

```

det_env = RollingBallEnv()
det_rewards = []
successes = 0
for ep in range(N_DET):
    obs, _ = det_env.reset(seed=ep)
    total_r = 0.0
    while True:
        action, _ = model.predict(obs, deterministic=True)
        obs, r, terminated, truncated, info = det_env.step(action)
        total_r += r
        if terminated or truncated:
            if terminated:
                successes += 1
            det_rewards.append(total_r)
            break
det_env.close()

print(f'Ewaluacja deterministyczna ({N_DET} epizodów):')
print(f' Skuteczność (dotarcie do celu): {successes/N_DET*100:.1f}%')
print(f' Średnia nagroda: {np.mean(det_rewards):.2f} ± {np.std(det_rewards):.2f}')
print(f' Min / Max: {min(det_rewards):.2f} / {max(det_rewards):.2f}')

all_configs = {**results, **arch_results}
ts, mean, std = aggregate(all_configs[best_config])

fig, ax = plt.subplots(figsize=(11, 5))
c = plt.cm.tab10.colors

ax.plot(ts, mean, color=c[0], linewidth=1.8,
        label=f'Trening - {best_config} (śr. z {N_SEEDS} seedów ± )')
ax.fill_between(ts, mean - std, mean + std, alpha=0.18, color=c[0])

det_mean = np.mean(det_rewards)
det_std = np.std(det_rewards)
ax.axhline(det_mean, color=c[1], linewidth=2.0, linestyle='-',
           label=f'Najlepszy agent (deterministycznie, {N_DET} ep.): {det_mean:.1f} ± {det_std:.1f} '
                f'({successes}/{N_DET} sukcesów)')
ax.axhspan(det_mean - det_std, det_mean + det_std, alpha=0.15, color=c[1])

ax.axhline(0, color='gray', linewidth=0.7, linestyle='--')
ax.set_xlabel('Kroki (timesteps)')
ax.set_ylabel('Nagroda epizodu')
ax.set_title(f'Porównanie: krzywa uczenia vs. deterministyczna ewaluacja\n'
             f'(najlepszy model: {best_config}, seed {best_seed})')

```

```

ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('best_agent_comparison.png', dpi=150)
plt.show()

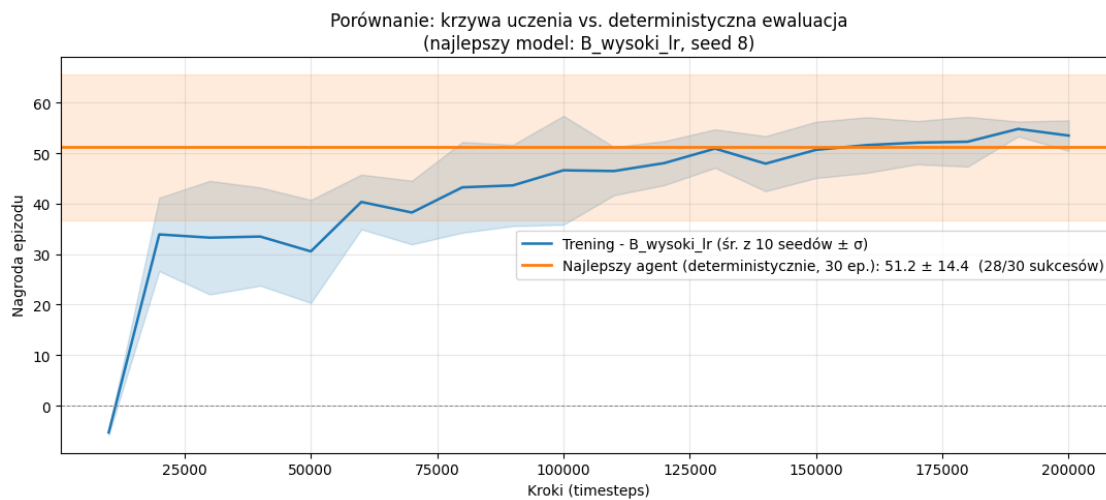
```

Ewaluacja deterministyczna (30 epizodów):

Skuteczność (dotarcie do celu): 93.3%

Średnia nagroda: 51.19 ± 14.40

Min / Max: $-2.78 / 60.24$



```

[ ]: # Symulacja na żywo w oknie pygame
model    = TD3.load(best_model_path)
live_env = RollingBallEnv(render_mode='human')

print(f'Symulacja na żywo - {best_config}, seed {best_seed} (10 epizodów)')
for ep in range(10):
    obs, _ = live_env.reset()
    while True:
        action, _ = model.predict(obs, deterministic=True)
        obs, _, terminated, truncated, _ = live_env.step(action)
        if terminated or truncated:
            break
    live_env.close()
print('Zakończono.')

```

Symulacja na żywo - B_wysoki_lr, seed 8 (10 epizodów)

Zakończono.

```
[20]: import imageio.v2 as imageio

model    = TD3.load(best_model_path)
rec_env  = RollingBallEnv(render_mode='rgb_array')

frames = []
for ep in range(5):
    obs, _ = rec_env.reset()
    frames.append(rec_env.render())
    while True:
        action, _ = model.predict(obs, deterministic=True)
        obs, _, terminated, truncated, _ = rec_env.step(action)
        frames.append(rec_env.render())
        if terminated or truncated:
            break
rec_env.close()

imageio.mimwrite('best_agent.mp4', frames, fps=30)
print(f'Zapisano best_agent.mp4 - 5 epizodów, {len(frames)} klatek,
      ↳ ({len(frames)/30:.1f} s @ 30 fps)')
```

IMAGEIO FFMPEG_WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (600, 600) to (608, 608) to ensure video compatibility with most codecs and players. To prevent resizing, make your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).

Zapisano best_agent.mp4 - 5 epizodów, 469 klatek (15.6 s @ 30 fps)

1.6 5. Wnioski

1. **TD3 w przestrzeni ciągłej** - algorytm skutecznie uczy się nawigacji w środowisku RollingBallEnv z ciągłą przestrzenią akcji. Najlepszy agent (konfiguracja **B**, seed 8) osiąga **93.3% skuteczności** dotarcia do celu w 30 deterministycznych epizodach testowych, ze średnią nagrodą 51.19 ± 14.40 . Mechanizm Twin Critics i opóźnione aktualizacje aktora zapewniają stabilne uczenie bez gwałtownych regresji polityki typowych dla DDPG.
2. **Wpływ hiperparametrów** - zarówno `learning_rate`, jak i `batch_size` istotnie wpływają na przebieg konwergencji. Konfiguracja **A** ($lr=3 \times 10^{-3}$, `batch=256`) osiąga 52.1 ± 2.8 przy 200 000 kroków i cechuje się najniższą wariancją między seedami - jest najstabilniejsza. Konfiguracja **B** ($lr=10^{-3}$, `batch=256`) osiąga porównywalną końcową nagrodę (53.5 ± 3.0) i generuje globalnie najlepszy pojedynczy model (B, seed 8, peak 56.63), jednak przy wyższej niestabilności w trakcie treningu ($=10.8$ przy 100 000 kroków vs $=6.8$ dla A). Konfiguracja **C** ($lr=10^{-3}$, `batch=128`) jest wyraźnie najslabsza - przy 200 000 kroków osiąga 46.0 ± 4.9 , czyli poziom, który A osiąga już przy 100 000 kroków.
3. **Wpływ architektury** - architektura duża [256, 256] (210 tys. parametrów) nieznacznie wygrywa końcową nagrodą (52.1 ± 2.8 vs 50.5 ± 3.6) i niższą wariancją. Zaskakującym wynikiem jest szybsza konwergencja architektury małej [64, 64] w środkowej fazie - przy

100 000 kroków mała sieć osiąga 50.4, przewyższając dużą (46.4). Różnica końcowa (~3%) sugeruje, że 13-wymiarowa obserwacja nie wymaga bardzo dużej pojemności sieciowej.

4. **Najlepszy model** - konfiguracja **B** (**lr=10⁻³**, **batch=256**), **seed 8**, checkpoint zapisany przez `EvalCallback` przy najwyższej treningowej nagrodzie ewaluacyjnej (56.63 z 10 epizodów). W deterministycznej ewaluacji na 30 epizodach osiągnął 51.19 ± 14.40 (93.3% sukcesów). Niższy wynik względem piku treningowego wynika z szerszej próby epizodów (30 vs 10) obejmującej trudniejsze losowe konfiguracje startowe.
5. **Kształtowanie nagrody** - gęsty sygnał $0.02 \cdot (\text{prev_dist} - \text{dist})$ jest kluczowy dla efektywnej konwergencji TD3. Bez niego agent musiałby uczyć się wyłącznie z rzadkiej nagrody +50, co dramatycznie wydłużyłoby czas treningu.